FlexQL: A Flexible SQL-like Database Driver

Objective

The objective of this assignment is to design and implement a simplified **SQL-like database driver** that supports a small subset of relational database functionality. The purpose of this project is to help students understand how database systems internally manage data storage, query execution, indexing, concurrency, and caching.

Students will implement a **client-server database system** in which the client provides an interactive terminal interface **(REPL)** for executing queries, while the server processes and executes the database operations.

The implementation must be done entirely in **C or C++**.

```
$ ./flexql-client 127.0.0.1 9000

Connected to FlexQL server

flexql> CREATE TABLE STUDENT(ID INT, NAME TEXT);

flexql> INSERT INTO STUDENT VALUES (1,'Alice');

flexql> INSERT INTO STUDENT VALUES (2,'Bob');

flexql> SELECT * FROM STUDENT;

ID = 1
NAME = Alice

ID = 2
NAME = Bob

flexql> .exit
Connection closed
```

System Overview

The system should consist of two components:

Client

The client should behave like a simple database terminal interface. It will allow the user to type SQL-like commands and send them to the server using the **Client C/C++ APIs** as described below.
From the user's perspective, the system should resemble the interaction style of command-line database tools.

Server

The server will implement the **database driver**. It will:

- Receive queries from the client
- Parse and validate the commands
- Execute operations on the database
- Return results to the client

The server must support **multiple simultaneous clients** and therefore should be implemented as a **multithreaded server**.

Supported SQL-like Commands

The database system must support only a limited subset of SQL operations.

a. CREATE TABLE

The system must allow users to create tables with defined columns.

Example format:

```
CREATE TABLE table_name (
    column1 TYPE,
    column2 TYPE,
    column3 TYPE,
    ...
);
```

The implementation should store the table schema and enforce it during insertion and querying. The "TYPE" should support:

- INT

- DECIMAL
- VARCHAR
- DATETIME

b. INSERT

The system must support inserting rows into a table.

Example format:

```
INSERT INTO table_name VALUES (...);
```

Each inserted row must also include an **expiration timestamp** (details described later).

c. SELECT

The system must support two forms of SELECT queries.

Select All Columns

```
SELECT * FROM table_name;
```

Select Specific Columns

```
SELECT column1, column2 FROM table_name;
```

d. WHERE Clause

The system must support a **single condition WHERE clause**.

Example:

```
SELECT * FROM table_name WHERE column = value;
```

Important restriction:

- Only **one condition** is allowed.
- Logical combinations such as **AND** or **OR** are **not to be supported**.

e. INNER JOIN

The system must support an **INNER JOIN between two tables** with a join condition.

Example format:

```
SELECT *
FROM tableA
INNER JOIN tableB
ON tableA.column = tableB.column;
```

The query may **include** a **WHERE clause** in addition to the join.

Storage Design

Students must design how the data is stored internally. The assignment intentionally leaves this decision to the students.

Students must decide:

- Whether the database should store data in row-major format or column-major format
- How tables and records will be represented in memory
- How schema information will be stored and accessed

The reasoning behind the chosen design must be explained in the documentation.

Indexing

The database must support **primary indexing** to improve query performance. Students must design an indexing structure for the primary key column. The specific data structure used for indexing is left to the students. The indexing mechanism should allow efficient lookup for queries involving the indexed column.

Caching for Performance

The system must implement a **caching mechanism** to improve query performance. The cache should be such that the system becomes more performant and repeated queries can be answered more quickly. Students must design the caching strategy. The strategies like LRU, LFU, or predictive. The choice is left to the students. The design of the caching mechanism and its impact on performance must be explained in the documentation.

Multithreaded Server

The database server must support **multiple concurrent client connections**. The server must therefore be implemented as a **multithreaded system** capable of processing multiple requests at the same time. Students must design mechanisms to ensure safe access to shared data structures. Appropriate concurrency control must be implemented to avoid race conditions and ensure correctness.

Performance Evaluation

The system will be evaluated using a dataset containing approximately **10 million rows**. The following metrics will be measured:

- Time required to **insert data**
- Time required to **execute SELECT queries**

Students should design their storage structures, indexing methods, and caching mechanisms with performance in mind. A script will be provided that will help you benchmark your driver on performance and the maximum memory footprint during benchmarking. There will be **a leaderboard** for both of these metrics.

Required Client C/C++ APIs

The following APIs must be implemented.

```
int flexql_open(const char *host, int port, FlexQL **db);

int flexql_close(FlexQL *db);

int flexql_exec(
    FlexQL *db,
    const char *sql,
    int (*callback)(void*, int, char**, char**),
    void *arg,
    char **errmsg
);

void flexql_free(void *ptr);
```

Database Handle

The database object must be defined as an **opaque structure**.

```
typedef struct FlexQL FlexQL;
```

The internal structure must not be exposed to the user.

API Specification

1. flexql_open

```
int flexql_open(const char *host, int port, FlexQL **db);
```

Description

Establishes a connection to the FlexQL database server.

Parameters

- **host*:*** Hostname or IP address of the database server. Ex: "127.0.0.1", "localhost", "192.168.1.10"
- **port:** Port number where the FlexQL server is listening. Example: 5432, 9000
- **db:** Pointer to a database handle that will be initialized by the function.

Return Values

- **FLEXQL_OK:** Connection closed successfully
- **FLEXQL_ERROR**: Invalid database handle

2. flexql_close

```
int flexql_close(FlexQL *db);
```

Description

Closes the connection to the FlexQL server and releases all associated resources.

Parameters

- **db:** Pointer to a database handle that will be initialized by the function.

Return Values

- **FLEXQL_OK:** Connection closed successfully
- **FLEXQL_ERROR**: Invalid database handle

Behavior

- Close the network connection
- Release memory associated with the database handle

3. flexql_exec

```
int flexql_exec(
  FlexQL* db,                              /* An open database */
  const char *sql,                         /* SQL to be evaluated */
  int (*callback)(void*,int,char**,char**), /* Callback function */
  void *arg,                               /* 1st argument to callback */
  char **errmsg                            /* Error msg written here */
);
```

Description

Executes an SQL statement on the FlexQL database server.
The SQL statement is sent to the server for execution.
If the query produces results (e.g., SELECT), the "**callback function**" must be invoked for each row returned.

Parameters

- **db:** Open database connection.
- **sql:** SQL statement to execute.
  Example:

  ```
  CREATE TABLE STUDENT(
      ID INT PRIMARY KEY NOT NULL,
      FIRST_NAME TEXT NOT NULL,
      LAST_NAME TEXT NOT NULL,
      EMAIL TEXT NOT NULL
  );
  ```

- **callback:** Function pointer invoked for each row of the query result.
  Prototype:

  ```
  int callback(void *data, int columnCount, char **values, char **columnNames);
  ```

  If callback is NULL, no result processing is required.
- **arg**: User-provided pointer passed to the callback function.
- **errmsg**: Pointer to an error message string if execution fails.

Return Values

- **FLEXQL_OK:** Connection closed successfully
- **FLEXQL_ERROR**: Invalid database handle

Callback Behavior

The callback function must be invoked once for every row returned by the query.

```
int callback(
    void *data,
    int columnCount,
    char **values,
    char **columnNames
);
```

Callback Parameters

- **data:** User data passed through the arg parameter.
- columnCount: Number of columns in the result row.
- values: Array of strings representing the column values.
  Example
  ```
  values[0] -> "1"
  values[1] -> "John"
  values[2] -> "Doe"
  values[3] -> "john@gmail.com"
  ```
- columnNames: Array containing column names.
  Example
  ```
  columnNames[0] -> "ID"
  columnNames[1] -> "FIRST_NAME"
  columnNames[2] -> "LAST_NAME"
  columnNames[3] -> "EMAIL"
  ```

Callback Return Value

- **0:** Continue processing
- **1**: Abort query execution

4. flexql_free

```
void flexql_free(void *ptr);
```

Description
Frees memory allocated by the FlexQL API.

Example

```
flexql_free(errMsg);
```

Error Codes

The following error codes must be defined.

```
#define FLEXQL_OK       0
#define FLEXQL_ERROR    1
```

Additional codes may be implemented if necessary.

Example Usage

```
#include <stdio.h>
#include <stdlib.h>
#include "flexql.h"

/* Callback function invoked for every row returned */
int callback(void *data, int columnCount, char **values, char **columnNames)
{
    for (int i = 0; i < columnCount; i++) {
        printf("%s = %s\n", columnNames[i], values[i] ? values[i] : "NULL");
    }

    printf("\n");
    return 0;
}
```

```c
int main()
{
    FlexQL *db;
    char *errMsg = 0;

    /* Connect to FlexQL server */
    int rc = flexql_open("127.0.0.1", 9000, &db);

    if (rc != FLEXQL_OK) {
        printf("Cannot connect to FlexQL server\n");
        return 1;
    }

    printf("Connected to FlexQL server\n");

    /* Create table */
    char *create_sql =
        "CREATE TABLE STUDENT("
        "ID INT PRIMARY KEY NOT NULL,"
        "FIRST_NAME TEXT NOT NULL,"
        "LAST_NAME TEXT NOT NULL,"
        "EMAIL TEXT NOT NULL);";

    rc = flexql_exec(db, create_sql, NULL, 0, &errMsg);

    if (rc != FLEXQL_OK) {
        printf("SQL error: %s\n", errMsg);
        flexql_free(errMsg);
    }

    /* Insert records */
    char *insert_sql =
        "INSERT INTO STUDENT VALUES(1,'John','Doe','john@gmail.com');";

    rc = flexql_exec(db, insert_sql, NULL, 0, &errMsg);

    if (rc != FLEXQL_OK) {
        printf("SQL error: %s\n", errMsg);
        flexql_free(errMsg);
    }
```

```c
    char *insert_sql2 =
        "INSERT INTO STUDENT VALUES(2,'Alice','Smith','alice@gmail.com');";

    rc = flexql_exec(db, insert_sql2, NULL, 0, &errMsg);

    if (rc != FLEXQL_OK) {
        printf("SQL error: %s\n", errMsg);
        flexql_free(errMsg);
    }

    /* Query data */
    char *select_sql = "SELECT * FROM STUDENT;";

    rc = flexql_exec(db, select_sql, callback, 0, &errMsg);

    if (rc != FLEXQL_OK) {
        printf("SQL error: %s\n", errMsg);
        flexql_free(errMsg);
    }

    printf("Query executed successfully\n");

    /* Close connection */
    flexql_close(db);

    return 0
}
```

Expected Output:

If the table contains two records, the program may print something like:

```
Connected to FlexQL server

ID = 1
FIRST_NAME = John
LAST_NAME = Doe
EMAIL = john@gmail.com

ID = 2
FIRST_NAME = Alice
LAST_NAME = Smith
EMAIL = alice@gmail.com

Query executed successfully
```

Implementation Constraints

The following constraints apply to the project:

- Programming language: **C or C++ only**
- External database libraries are **not allowed**
- The entire query handling and storage logic must be implemented by the students
- The client's interactive terminal interface (REPL) **must** use the APIs described above

Deliverables

Students must submit the following:

1. Source code of the complete system on **GitHub** (Add link to repository at the start of the design document that will be submitted on Moodle)
2. A design document describing:
    - How the data is stored
    - Indexing method
    - Caching strategy
    - Handling of expiration timestamps
    - Multithreading design
    - And any other design decisions that you have made
3. Compilation and execution instructions
4. Performance results for large datasets

NOTE

1. Absence of any component or not adhering to the submission guidelines will result in a penalty.
2. Sample Folder Structure (given on the next page, not necessary to follow it)

Sample Folder Structure

```bash
flexql/
├── bin/
├── build/
├── config/
├── scripts/
├── include/
│   ├── common/
│   ├── network/
│   ├── parser/
│   ├── query/
│   ├── storage/
│   ├── index/
│   ├── cache/
│   ├── expiration/
│   ├── concurrency/
│   └── utils/
├── src/
│   ├── client/
│   ├── server/
│   ├── network/
│   ├── parser/
│   ├, query/
│   ├── storage/
│   ├── index/
│   ├── cache/
│   ├── expiration/
│   ├── concurrency/
│   └── utils/
├── tests/
└── data/
    ├── tables/
    ├── indexes/
    └── wal/
```